

SAPIENZA
UNIVERSITÀ DI ROMA

Tailscale and NAT Traversal: An Experimental Analysis of Peer-to-Peer Connectivity Techniques Across NAT

Faculty of Information Engineering, Computer Science and Statistics
Bachelor's Degree in Computer Science

Alessia Cassetta
ID number 2113909

Advisor
Prof. Angelo Spognardi

Academic Year 2025/2026

Tailscale and NAT Traversal: An Experimental Analysis of Peer-to-Peer Connectivity Techniques Across NAT
Bachelor's thesis. Sapienza University of Rome

© 2026 Alessia Cassetta. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: alessiacassetta18@gmail.com

Abstract

Contents

1 Introduction **1**

2 Fundamentals **2**
2.1 IPv4 and Private Networks 2
2.1.1 Public and Private IPv4 Addresses 3
2.2 Network Address Translation 4
2.2.1 NAT and NAPT 4
2.2.2 Mapping and Filtering Behavior 5
2.2.3 NAT Limitations 6
2.3 Peer-to-Peer Connectivity Across NAT 7
2.4 NAT Traversal 8
2.4.1 UDP Hole Punching 8
2.4.2 STUN 10
2.4.3 Relaying and TURN 10
2.4.4 ICE 11
2.5 VPN Architectures 12
2.5.1 Centralized VPN 12
2.5.2 Peer-to-Peer VPNs 12
2.5.3 Control Plane and Data Plane 13
2.6 WireGuard 14
2.6.1 Peer Configuration and Secure Communication 14
2.6.2 Connectivity Across NAT 15

3 Tailscale **17**
3.1 Tailscale Overview 17
3.2 Tailscale Architecture 17
3.3 Peer Discovery and Endpoint Management 17
3.4 Connection Establishment 17
3.5 Relayed Connections 17
3.6 Tailscale Diagnostic Tools 17

4 Experimental Analysis **18**
4.1 Experimental Environment 18
4.2 Experimental Methodology 18
4.3 Direct Connectivity Experiments 18
4.4 Relayed Connectivity Experiments 18
4.5 Connection Path Evolution 18
4.6 Results and Discussion 18

5 Conclusions **19**

Acknowledgements **20**

Bibliography **21**

Chapter 1

Introduction

Chapter 2

Fundamentals

2.1 IPv4 and Private Networks

Definition 2.1. The *Internet Protocol* (IP) allows devices connected to a network to be identified and packets to be routed between different networks.

An IPv4 address is 32 bits long, equivalently 4 bytes, which gives a total of 2^{32} possible addresses. IPv4 addresses are written using *dotted-decimal notation*, where the four bytes are represented as decimal numbers separated by dots, such as `192.168.1.10` [RFC 791].

An IPv4 address identifies not just a device, but, more precisely, one of its interfaces. A host that has multiple interfaces can have multiple IPv4 addresses simultaneously. Every IPv4 address is logically divided into a part that identifies the network and a part that identifies the host within that network. This division is represented by the *network prefix*, normally expressed in *CIDR notation* [RFC 4632].

Definition 2.2. A *subnet* is a logical portion of an IP network identified by a common prefix.

***Example* 2.1.** The address `192.168.1.10/24` indicates that the first 24 bits of the address are used to identify the network, while the remaining 8 bits are used to identify the host within that network. In this case, the network prefix is `192.168.1.0/24`, and the hosts in this network can communicate with each other directly.

Instead, when the destination belongs to a different network, the packet must be forwarded to a router.

Definition 2.3. A *router* is a network device which links two or more IP networks and forwards datagrams between them based on their destination IP address.

A host uses its routing table to determine where to forward a packet. If the path to the destination doesn't already exist, the traffic is forwarded to a default gateway, a router used to reach external networks.

A sample topology is shown in Figure 2.1.

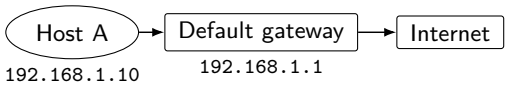


Figure 2.1. Example of a host communicating with an external network through its default gateway.

In this example, `192.168.1.1` represents the default gateway for the host to communicate with destination addresses outside the local network.

2.1.1 Public and Private IPv4 Addresses

Not all IPv4 addresses are globally routable.

- *Public addresses* are globally unique and can be used to identify an interface on the public Internet. An Internet router can thus use its own routing information to forward the traffic to the prefix to which a specific public address belongs.
- *Private addresses*, instead, are destined to be used only within a private network. They are not globally unique and cannot be used to identify an interface on the public Internet.

For example, two hosts that belong to different private networks can have the same private address, such as `192.168.1.10`, without any conflict, since they are not reachable from the public Internet [RFC 1918].

This characteristic allows the same private address space to be reused in different networks, reducing the number of public IPv4 addresses.

However, it also has an important implication: a private address does not uniquely identify a device globally. A mechanism is needed to map the private addressing space to the public addressing space. One of the most commonly used mechanisms to perform this function is Network Address Translation (NAT), which will be described in the next section.

NAT allows the addresses used within the private network to be replaced with addresses valid in the external addressing space, while retaining the information necessary to correctly forward the response traffic.

2.2 Network Address Translation

Definition 2.4. *Network Address Translation (NAT)* is a method used by routers to let multiple devices on a private local network share a single public IP address to access the internet.

NAT is commonly deployed at the boundary between a private IPv4 network and the Internet [RFC 2663]. Hosts inside the private network use addresses that are not globally routable, while the NAT device translates these addresses into addresses that can be used in the external network.

When a host initiates a communication, the NAT creates a binding between the internal address and an address belonging to the external address realm. This binding allows packets traveling between the two networks to be translated consistently in both directions during the lifetime of the communication.

***Example* 2.2.** An internal host using the address `192.168.1.10` may be associated with the public address `203.0.113.20` when communicating with hosts on the Internet.

Address bindings can be managed using either static or dynamic address assignment.

- In *static* assignment, a private address is permanently associated with a specific external address for the lifetime of the NAT configuration. This type of assignment ensures that NAT does not have to supervise address management in a session.
- In *dynamic* assignment, addresses are assigned to private network hosts dynamically. When the last session using an address binding is terminated, NAT frees the binding so that the global address can be reused later.

2.2.1 NAT and NAPT

One of the most common scenarios involves connecting a network that uses private IPv4 addresses to the Internet. The addresses that are in private space are not globally routable. Thus, they need to be translated before the host communicates with a device on the public network.

The first form of translation is *Basic NAT*, where only the IP address is modified. An address belonging to a private network is associated with an address of a public domain.

In a modern network, however, *Network Address Port Translation (NAPT)* is commonly used. It extends the translation mechanism, modifying not only the IP address but also the transport protocol identifier, typically the TCP or UDP port number. In this way, more devices in the private network can share just one public IPv4 address simultaneously [RFC 3022].

***Example* 2.3.** Let's consider a host with an internal endpoint `192.168.1.10:5000` which sends a UDP datagram to the Internet. The NAPT device could create the following association:

$$\texttt{192.168.1.10:5000} \longleftrightarrow \texttt{203.0.113.20:40000}$$

From the point of view of the external host, the traffic does not come from `192.168.1.10:5000`, but from the public endpoint `203.0.113.20:40000`.

The NAT must maintain a *state*, or mapping, which allows it to map the packets received on the external endpoint to the correct host in the private network. In conclusion, a response directed to `203.0.113.20:40000` can be translated again and forwarded to `192.168.1.10:5000`.

This behavior makes it relatively easy to initiate communication from within the private network, since sending the first packet allows the NAT to create the necessary state.

On the contrary, communication initiated from the external network is more difficult because no mapping may exist yet. So the NAT doesn't have enough information to determine which internal host should receive the packet.

2.2.2 Mapping and Filtering Behavior

To comprehend the NAT Traversal techniques, it is not enough to know that NAT modifies the IP address and port number. It's important to analyze the mapping and filtering behavior of the NAT device separately.

This distinction is formalized in [RFC 4787], which describes the behavioral requirements for NATs handling UDP traffic.

Mapping Behavior The mapping behavior determines in which way the NAT associates an internal endpoint with an external one. Let's consider the internal endpoint `192.168.1.10:5000`. When this device communicates with an external server, the NAT can assign to it the public endpoint `203.0.113.20:40000`.

The fundamental question is what happens when the same internal endpoint communicates with a different external destination. And we can distinguish three different behaviors:

- In the *Endpoint-Independent Mapping* behavior, the same internal endpoint maintains the same external mapping regardless of the external destination. For example, a host can contact Server A and Server B while retaining the same mapped public endpoint, as shown in Table 2.1. This behavior is particularly useful for NAT Traversal techniques, since the public endpoint tends to be reused to communicate with a different peer.

Table 2.1. Example of endpoint-independent mapping for two remote destinations.

Internal endpoint	Destination	External endpoint
`192.168.1.10:5000`	Server A	`203.0.113.20:40000`
`192.168.1.10:5000`	Server B	`203.0.113.20:40000`

- In the *Address-Dependent Mapping* behavior, the mapping depends on the destination IP address. The same internal endpoint can be associated with different external endpoints when communicating with different external destinations.

- In the *Address and Port-Dependent Mapping* behavior, the mapping depends on both the destination IP address and the remote endpoint's port number. In this case, the public endpoint observed during communication with a specific server may not be the one used by NAT when communicating with another peer. This behavior makes NAT traversal more complex, because knowing the observed mapping to an external server does not necessarily guarantee that one can predict the mapping that will be used when communicating with another device.

Filtering Behavior The filtering behavior, on the other hand, concerns the packets coming from the external network and determines which sources are allowed to reuse the existing mapping.

Filtering is distinct from mapping: two NATs can assign the same external endpoint in the same way, but they can accept the traffic with different filtering rules.

- In *Endpoint-Independent Filtering*, once the mapping has been created, NAT can accept traffic directed to that mapping regardless of the external sender's address and port.
- In *Address-Dependent Filtering*, NAT allows traffic only from IP addresses to which the internal host has previously sent traffic.
- Finally, in *Address and Port-Dependent Filtering*, packets are accepted only from the exact remote endpoint—that is, the specific IP address and port pair—to which the internal host has previously sent traffic.

This distinction is important in peer-to-peer communication. An application that wants to establish a direct connection needs to consider both the public endpoint assigned by the NAT and the necessary conditions, so that packets from the peer can be accepted and forwarded to the internal host.

2.2.3 NAT Limitations

Although NAT is generally transparent to many client-server applications, address translation violates the principle that an endpoint can be consistently identified by its IP address.

[RFC 3027] examines several scenarios in which the presence of a NAT can interfere with the proper functioning of protocols and applications.

The first issue concerns *the applications which include the IP address or port number in the payload of the packet.* An ordinary NAT modifies only the IP header and, in the case of NAPT, also the transport header. It does not know the contents of the payload.

***Example* 2.4.** Consider an application that includes the following information in the payload of a packet:

```
Connect to 192.168.1.10:5000
```

Even if the NAT correctly translates the source IP address in the packet header, the private address contained in the payload remains unchanged.

Consequently, an external host cannot use this address to establish a connection with the internal device.

A possible solution is to use an *Application Layer Gateway (ALG)*. An ALG knows the specific application protocol and can modify the information in the payload. However, it requires that the intermediate device understand the application protocol and maintain the necessary state.

The second issue concerns *bundled session applications*, that is, applications in which a connection is used to negotiate additional connections. FTP, a standard network protocol used to transfer files between a local computer and a remote server on the Internet or on a corporate network, is a classic example: a control connection can contain information about both the address and port to be used for another data connection. The NAT can know the relation between those two sessions.

Another important category is *peer-to-peer applications*. Unlike the client-server model, in which the server has a known endpoint and it is reachable, in a peer-to-peer application both nodes may need to initiate communication. If one or both devices are under NAT, the private addresses are not reachable from the public network.

Additional issues relate to *IP fragmentation*, since fragments following the first one may not contain the transport headers necessary for NAPT to correctly identify the translation to be performed.

Some applications also require *the same address mapping to be maintained for long periods of time*. Since NAT associations are temporary, they may expire in the absence of traffic. For this reason, some applications use periodic keep-alive messages.

Finally, protocols that use *integrity checks or encryption* may be incompatible with certain forms of translation. If the information that a NAT or ALG is supposed to modify is cryptographically protected, the intermediate device may not be able to alter it without invalidating the protection.

2.3 Peer-to-Peer Connectivity Across NAT

The *peer-to-peer* model differs from the *client-server* model because each node can act as both a sender and a recipient in a communication.

In a client-server architecture, there is a server with a known and accessible address: `Client` $\longrightarrow$ `Server`. Even if the client is located behind NAT, it can establish a connection to the server. NAT then creates the necessary mapping and allows packets belonging to the session to be returned.

The scenario becomes more complex when both peers are located within private networks, as shown in Figure 2.2.

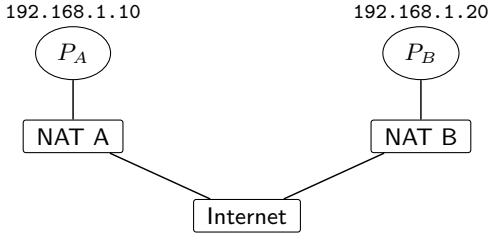


Figure 2.2. Two peers located behind different NAT devices.

P_B's IP address `192.168.1.20` does not identify the device on the public network and could also be used simultaneously by other hosts belonging to different private networks.

Thus, P_A needs to know the public endpoint associated with P_B, represented by its public IP address and port number.

However, knowing such an endpoint is not always enough. NAT B needs to maintain a state that allows packets from P_A to pass through it. The same problem exists in the opposite direction.

To establish direct communication, at least two conceptual elements are therefore necessary:

- A mechanism through which the peers can discover and exchange their respective external endpoints.
- A mechanism that allows the necessary state to be created in the NAT, so that traffic from the other peer can be accepted.

And from this problem arise the main *NAT traversal* techniques.

2.4 NAT Traversal

Definition 2.5. *NAT Traversal* is a set of techniques used to allow communication between endpoints separated by one or more NATs, particularly when a communication needs to be initiated from the external network or directly between two peers.

The problem can be addressed in different ways. You can explicitly configure NAT using static mapping or port forwarding, use application gateways, or employ techniques in which peers dynamically discover their connectivity information and attempt to establish a direct path.

When a direct connection cannot be initiated, it is also possible to use a *relay* that is, an intermediate node through which traffic is routed.

2.4.1 UDP Hole Punching

Definition 2.6. *UDP hole punching* is one of the main techniques used to establish direct communication between peers located behind different NATs.

Consider two nodes, P_A and P_B, both of which belong to private networks (Figure 2.3).

Since the private addresses of the two peers are not globally routable, P_A and P_B cannot use them directly to initiate communication over the Internet.

In the first step, both peers contact a publicly accessible server, often referred to as a *rendezvous server*. Sending traffic to this server triggers the creation of a mapping in their respective NATs.

The rendezvous server can then observe the public endpoints of the two peers and allow them to exchange this information:

$$\texttt{A} \rightarrow \texttt{203.0.113.10:40000}$$
$$\texttt{B} \rightarrow \texttt{198.51.100.20:45000}$$

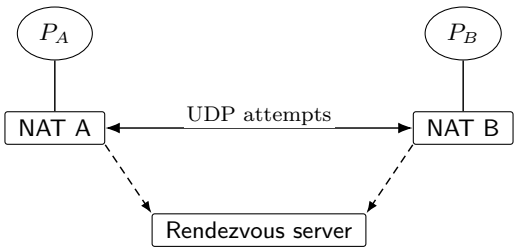


Figure 2.3. Conceptual UDP hole punching scenario. Dashed arrows indicate initial communication with the rendezvous server; the horizontal path represents the attempted direct connection.

Once it has learned the other peer's public endpoint, P_A begins sending datagrams to `198.51.100.20:45000`, while P_B sends datagrams to `203.0.113.10:40000`. The first few packets may be discarded. However, the outgoing traffic helps establish the necessary state in the respective NATs to subsequently allow traffic from the other peer.

The desired final result is a direct UDP connection between P_A and P_B, without the application traffic having to permanently pass through an intermediate server.

Outgoing traffic creates or updates the state in the respective NATs. In this way, each NAT can accept packets from the other peer. The first datagrams may be discarded if the necessary state does not yet exist, but subsequent attempts may allow direct communication to be established.

The success of UDP hole punching depends on the behavior of the NATs involved [RFC 5128]. An *Endpoint-Independent Mapping* facilitates the process, since the public endpoint observed during communication with the rendezvous server can also be reused for communication with the peer.

***Example* 2.5.** Consider a NAT that assigns different external mappings depending on the destination address.

When a peer contacts a rendezvous server, the NAT creates a mapping that allows the server to observe a public endpoint.

However, when the same peer attempts to communicate with another peer, the NAT may assign a different external mapping.

As a result, the endpoint discovered through the rendezvous server may not correspond to the one used for direct peer-to-peer communication, potentially preventing successful UDP hole punching.

Finally, since UDP mappings generally have a limited duration, peers may periodically send keep-alive packets to prevent the state maintained by the NAT from expiring.

UDP hole punching provides only a connectivity mechanism and does not guarantee properties such as authentication, confidentiality, or integrity. These properties must be provided separately by the protocol used for communication.

2.4.2 STUN

Definition 2.7. *STUN (Session Traversal Utilities for NAT)* is a protocol used to support other protocols and applications that need to operate in the presence of NAT.

One of its main uses is to allow a host located behind NAT to determine the endpoint through which it is visible to the external network [RFC 8489].

***Example* 2.6.** Consider a client located behind a NAT with the following private endpoint: `192.168.1.10:5000` When the client communicates with an external STUN server, the NAT creates a mapping between the private endpoint and a public endpoint:

$$\texttt{192.168.1.10:5000} \longrightarrow \texttt{203.0.113.10:40000}$$

The client sends a *STUN Binding Request* to the server, which observes the public source IP address and port.

The server returns this information in a *Binding Success Response*.

The client can therefore discover that its private endpoint is externally visible as `203.0.113.10:40000`.

This address is referred to as the *server-reflexive transport address*.

This information can be used as part of a NAT traversal procedure. For example, two peers can use STUN to determine their respective external endpoints, exchange this information through a coordination mechanism, and then attempt to establish direct communication via UDP hole punching.

It is important to note that STUN alone does not constitute a complete solution for NAT traversal. The protocol allows one to observe the mapping created by NAT for a specific communication, but it does not guarantee that the same mapping will be reused for different destinations.

2.4.3 Relaying and TURN

When a direct communication between two peers cannot be established, an alternative approach is to route data through a publicly accessible node.

This approach eliminates the necessity for the two peers to be directly reachable to each other: both must be able to establish an outgoing connection to the relay.

Definition 2.8. *TURN (Traversal Using Relays around NAT)* standardizes this type of mechanism and is defined as an extension of STUN that allows a client to use an intermediate server to exchange traffic with one or more peers.

The client requests the allocation of an endpoint on the relay from the TURN server. The server then provides a *relayed transport address* usable to receive the traffic destined for the client [RFC 8656]. Figure 2.4 illustrates the logical relay path.



Figure 2.4. Logical communication path through a TURN relay.

The principal benefit of using a relay is the higher probability to work even with the presence of particularly restrictive the NATs or firewalls.

However, it exists a cost. All traffic pass through the intermediate server, increasing the bandwidth usage of the infrastructure and the latency compared to a direct route Thats why, relay is generally used as a *fallback* when is not possible to establish a direct connection.

It is important to separate the general concept of a relay from the TURN protocol: different systems can implements their own relay infrastructures without using TURN.

2.4.4 ICE

Definition 2.9. *ICE (Interactive Connectivity Establishment)* is a NAT traversal protocol and it uses STUN and TURN to identify different communication possibilities between two endpoint and verify which of them are actually working [RFC 8445].

The main idea behind ICE is to avoid assuming in advance which network path will be available. Instead, each peer gathers a set of possible addresses, called *candidates*, through which it may be reachable.

The main categories include:

- *Host Candidate*: an address directly associated with one of the local network interfaces of the peer;
- *Server-Reflexive Candidate*: an external address discovered through STUN, representing the endpoint assigned by a NAT;
- *Relayed Candidate*: an address allocated by a TURN server and used when communication must pass through a relay;

During the connectivity-checking process, ICE may also discover *Peer-Reflexive Candidates*, which are candidate addresses learned dynamically as a consequence of STUN connectivity checks.

After gathering the available candidates, the peers exchange them and form *candidate pairs*, where each pair represents a possible communication path between a local candidate and a remote candidate.

ICE then performs *connectivity checks* on these candidate pairs. These checks are based on STUN and are used to determine whether packets can successfully travel between the two corresponding endpoints. Once a valid candidate pair has been identified, it can be selected and nominated for carrying the communication.

ICE therefore operates at a higher logical level than STUN and TURN. STUN is mainly used for endpoint discovery and connectivity checks, while TURN provides a relay when a direct path is not available. ICE coordinates these mechanisms and selects a working communication path. Thus, STUN, TURN, and ICE should not be considered as three independent alternatives. Instead, they can work together as parts of the same NAT traversal strategy.

2.5 VPN Architectures

Definition 2.10. A *Virtual Private Network (VPN)* allows the creation of a logical private network using an IP infrastructure that can be shared or public. In this way, devices located in a different networks can communicate as if they were part of the same private network.

The [RFC 2764] describes a general framework for IP-based VPNs and presents a VPN as a mechanism for emulating a private network over an IP infrastructure.

It's helpful to distinguish between the different ways in which connectivity between devices is established.

2.5.1 Centralized VPN

In a *Centralized VPN*, devices generally establish a tunnel to a central server or gateway. Traffic between two clients is then forwarded through this node.

A simplified representation is shown in the following Figure: 2.5.

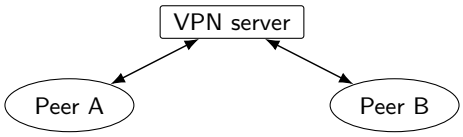


Figure 2.5. Block diagram of a centralized VPN: traffic between peers passes through the VPN server.

This type of model has several pros. Central server typically has a stable and reachable public IP address, allowing clients to initiate outbound connection to it. As a result, the presence of NAT on clients is easier to menage compared with a scenario where two devices located under different NATs must establish a direct connection.

Centralized infrastructure can simplify various management functions, such as user and device authentication, network policy enforcement and overall VPN administration.

However, in this architecture, the central server is also part of the path taken by application traffic. Packets must therefore pass through it even when the two devices could, from a network perspective, communicate directly.

This can result in increased latency, higher bandwidth consumption on the central infrastructure, and greater reliance on the server for communications to function properly.

2.5.2 Peer-to-Peer VPNs

An alternative approach involves using a *peer-to-peer* architecture, in which nodes attempt, whenever possible, to communicate directly with one another.

Figure 2.6 shows an example of possible logical connections between peers:

The absence of an intermediate node in the data path can reduce dependence on a centralized routing infrastructure and, when network conditions permit, facilitate shorter communication paths.

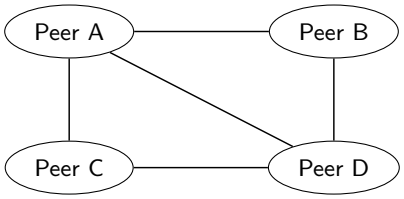


Figure 2.6. Example of direct logical connections in a peer-to-peer VPN. It is not necessary for every pair of nodes to have an active connection.

However, the ability to establish a direct connection depends on the reachability of the endpoints and on the restrictions imposed by NATs and firewalls along the path.

For this reason, peer-to-peer VPNs operating over the Internet may require endpoint discovery mechanisms and NAT traversal techniques—including UDP hole punching—as discussed in the previous sections. If direct communication is not possible, the architecture may involve the use of a relay node to transport traffic.

It is important to note that the peer-to-peer model does not necessarily imply completely decentralized management of the VPN. Even when data is transmitted directly between nodes, a central service can perform functions such as coordination, configuration distribution, and access control.

2.5.3 Control Plane and Data Plane

In network architectures, the *control plane* comprises the mechanisms by which the information necessary to establish and manage connectivity is defined and distributed.

The *data plane*, also known as the *forwarding plane*, comprises the functions that actually process and forward packets.

In the context of an overlay VPN, the control plane may be responsible for tasks such as:

- Identification and authorization of nodes participating in the VPN;
- Distribution of information regarding peers and their endpoints;
- Configuration of the parameters required to create tunnels;
- Distribution of policies governing communication between nodes.

The data plane, on the other hand, is responsible for transporting the traffic generated by users and applications. Depending on the technology used, these operations may include encapsulation, encryption, decryption, and forwarding of packets through the VPN connections.

The two components therefore perform different functions: the control plane provides the information necessary for the network to operate, while the data plane carries out data transfer according to the established configuration.

Figure 2.7 illustrates a scenario in which a central infrastructure coordinates two peers, which communicate directly over the data plane.

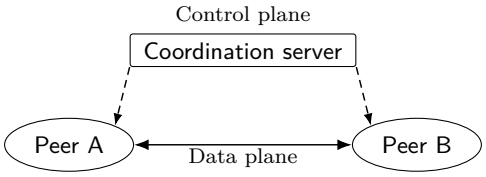


Figure 2.7. Separation between the centralized control plane (dashed lines) and the direct data plane between peers (solid line).

In this scenario, the nodes obtain the information needed to configure the VPN from the central service, but application traffic follows a direct path between the endpoints.

The centralization of the control plane therefore does not necessarily entail the centralization of the data plane.

The separation between the two planes also makes it possible to distinguish coordination decisions from the actual path taken by packets. The latter may vary depending on connectivity conditions: for example, a direct connection may be preferred when available, while a route via a relay may be used when network restrictions prevent peer-to-peer communication.

2.6 WireGuard

Definition 2.11. *WireGuard* is a Layer 3 VPN protocol designed to provide secure IP communication through encrypted tunnels. Its design focuses on simplicity, performance and a limited attack surface. It uses UDP as its transport protocol and relies on public-key cryptography to authenticate peers and protect the exchanged traffic.

Unlike traditional VPN solutions that may include additional mechanisms for user management and network coordination, WireGuard focuses on establishing secure communication between configured peers.

2.6.1 Peer Configuration and Secure Communication

Each WireGuard interface has its own private key, while remote peers are identified through their public keys. To establish communication, a peer must be configured with information such as:

- Its own private key;
- The public key of the remote peer;
- The IP addresses or prefixes associated with that peer;
- The remote endpoint, typically an IP address and UDP port, when required.

A fundamental feature of WireGuard is *Cryptokey Routing*, which associates each peer's public key with a set of IP addresses or network prefixes, known as *AllowedIPs*.

This mechanism serves two purposes. For outgoing traffic, AllowedIPs determines which peer should receive a packet based on its destination IP address. For incoming

traffic, it acts as an access-control mechanism, ensuring that an authenticated peer can only send packets with source addresses permitted by its configuration.

***Example* 2.7.** Consider two WireGuard peers, P_A and P_B.

P_A is configured with P_B's public key and the following AllowedIPs entry:

```
AllowedIPs = 10.0.0.2/32
```

When a packet destined for `10.0.0.2` is sent through P_A's WireGuard interface, the protocol selects P_B according to the configured AllowedIPs.

The packet is then encrypted, encapsulated inside a UDP datagram and transmitted to P_B's known endpoint.

After receiving the datagram, P_B authenticates and decrypts its contents. If the source IP address of the original packet is permitted by P_B's configuration, the packet is accepted and delivered through the virtual network interface.

Before encrypted data can be exchanged, WireGuard performs a cryptographic handshake based on the Noise protocol framework. This process allows the peers to authenticate each other and establish temporary symmetric keys used to protect subsequent communication.

These session keys are periodically renewed, providing forward secrecy and limiting the consequences of a potential key compromise.

2.6.2 Connectivity Across NAT

Although WireGuard provides the mechanisms needed to secure communication, it does not directly handle all the operations required to establish connectivity between peers.

This limitation becomes particularly relevant when devices are located behind NAT.

Consider the topology previously illustrated in Figure 2.2, where two peers belong to different private networks and communicate through separate NAT devices.

WireGuard can encrypt and authenticate the exchanged packets, but it does not automatically determine how the peers can reach each other across those NATs.

In a manually configured deployment, a peer may be assigned a fixed remote endpoint, consisting of an IP address and UDP port. This approach works when the endpoint is reachable, but becomes less reliable when public addresses change or when both peers are located behind restrictive NATs.

To accommodate changes in network connectivity, WireGuard supports a mechanism known as *endpoint roaming*.

When a peer receives a valid authenticated packet from a different source IP address or UDP port, WireGuard can update the corresponding remote endpoint. Subsequent packets can then be sent to this newly observed address.

This behavior is useful when a device changes networks or when its public endpoint changes due to NAT. However, endpoint roaming depends on receiving an authenticated packet from the remote peer. It does not provide a general mechanism for discovering an initially unreachable endpoint or establishing connectivity across different NATs.

Another feature relevant to NAT environments is *PersistentKeepalive*.

WireGuard normally avoids transmitting unnecessary packets when no application traffic is being exchanged. While this reduces network overhead, it can create difficulties when a peer is behind a NAT or stateful firewall.

Since UDP mappings are generally temporary, an inactive connection may lose the mapping that allows incoming packets to reach the internal device.

The `PersistentKeepalive` option addresses this problem by periodically transmitting empty packets to the remote endpoint, helping maintain the NAT or firewall state.

However, this mechanism should not be confused with NAT traversal. Persistent keepalive packets can preserve an existing mapping, but they do not discover the public endpoint of a peer or guarantee that a direct connection can be established.

WireGuard therefore provides the secure tunneling mechanism used to transport data between peers, while other components may be necessary to discover endpoints, exchange connection information and establish communication across NATs.

Chapter 3

Tailscale

3.1 Tailscale Overview

3.2 Tailscale Architecture

3.3 Peer Discovery and Endpoint Management

3.4 Connection Establishment

3.5 Relayed Connections

3.6 Tailscale Diagnostic Tools

Chapter 4

Experimental Analysis

4.1 Experimental Environment

4.2 Experimental Methodology

4.3 Direct Connectivity Experiments

4.4 Relayed Connectivity Experiments

4.5 Connection Path Evolution

4.6 Results and Discussion

Chapter 5

Conclusions

The conclusions of the thesis are presented here.

Acknowledgements

Bibliography

[RFC 791] J. Postel, *Internet Protocol*, RFC 791, Information Sciences Institute, September 1981.

[RFC 1918] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear, *Address Allocation for Private Internets*, RFC 1918, Internet Engineering Task Force, February 1996.

[RFC 4632] V. Fuller and T. Li, *Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan*, RFC 4632, Internet Engineering Task Force, August 2006.

[RFC 2663] P. Srisuresh and M. Holdrege, *IP Network Address Translator (NAT) Terminology and Considerations*, RFC 2663, Internet Engineering Task Force, August 1999.

[RFC 3022] P. Srisuresh and K. Egevang, *Traditional IP Network Address Translator (Traditional NAT)*, RFC 3022, Internet Engineering Task Force, January 2001.

[RFC 4787] F. Audet and C. Jennings, *Network Address Translation (NAT) Behavioral Requirements for Unicast UDP*, RFC 4787, Internet Engineering Task Force, January 2007.

[RFC 3027] M. Holdrege and P. Srisuresh, *Protocol Complications with the IP Network Address Translator*, RFC 3027, Internet Engineering Task Force, January 2001.

[RFC 5128] P. Srisuresh, B. Ford, and D. Kegel, *State of Peer-to-Peer (P2P) Communication across Network Address Translators (NATs)*, RFC 5128, Internet Engineering Task Force, March 2008.

[RFC 8489] M. Petit-Huguenin, G. Salgueiro, J. Rosenberg, D. Wing, R. Mahy, and P. Matthews, *Session Traversal Utilities for NAT (STUN)*, RFC 8489, Internet Engineering Task Force, February 2020.

[RFC 8656] T. Reddy, A. Johnston, P. Matthews, and J. Rosenberg, *Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN)*, RFC 8656, Internet Engineering Task Force, February 2020.

[RFC 8445] A. Keranen, C. Holmberg, and J. Rosenberg, *Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal*, RFC 8445, Internet Engineering Task Force, July 2018.

[RFC 2764] B.G̃leeson, A.L̃in, J.H̃einanen, G.Ãrmitage, and A.M̃alis, *A Framework for IP Based Virtual Private Networks*, RFC 2764, Internet Engineering Task Force, February 2000.

[WireGuard] J. A. Donenfeld, *WireGuard: Next Generation Kernel Network Tunnel*, Technical Whitepaper, June 2020.

[WireGuard Protocol] WireGuard Project, *Protocol & Cryptography*, Official Documentation, accessed October 23, 2026.

[WireGuard Quick Start] WireGuard Project, *Quick Start*, Official Documentation, accessed October 23, 2026.